

Experiment 3: Playfair Cipher

Program :

```
#include <stdio.h> #include <string.h> #include <ctype.h>

char matrix[5][5];

void generateMatrix(char *key) { int used[26] = {0}; char letters[25]; int k = 0;

for (int i = 0; key[i] != '\0'; i++) {
    char ch = toupper(key[i]);

    if (ch == 'J')
        ch = 'I';

    if (ch >= 'A' && ch <= 'Z' && !used[ch - 'A']) {
        used[ch - 'A'] = 1;
        letters[k++] = ch;
    }
}

for (char ch = 'A'; ch <= 'Z'; ch++) {
    if (ch == 'J')
        continue;

    if (!used[ch - 'A']) {
        used[ch - 'A'] = 1;
        letters[k++] = ch;
    }
}

k = 0;

for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        matrix[i][j] = letters[k++];
    }
}
```

```
    }  
}
```

```
}
```

```
void findPosition(char ch, int *row, int *col) { if (ch == 'J') ch = 'I';
```

```
    for (int i = 0; i < 5; i++) {  
        for (int j = 0; j < 5; j++) {  
            if (matrix[i][j] == ch) {  
                *row = i;  
                *col = j;  
                return;  
            }  
        }  
    }  
}
```

```
}
```

```
void processPair(char a, char b, char *x, char *y, int encrypt) { int r1, c1, r2, c2;
```

```
    findPosition(a, &r1, &c1);
```

```
    findPosition(b, &r2, &c2);
```

```
    if (r1 == r2) {  
        if (encrypt) {  
            *x = matrix[r1][(c1 + 1) % 5];  
            *y = matrix[r2][(c2 + 1) % 5];  
        } else {  
            *x = matrix[r1][(c1 + 4) % 5];  
            *y = matrix[r2][(c2 + 4) % 5];  
        }  
    }
```

```
}
```

```
    else if (c1 == c2) {  
        if (encrypt) {  
            *x = matrix[(r1 + 1) % 5][c1];  
            *y = matrix[(r2 + 1) % 5][c2];
```

```

    } else {
        *x = matrix[(r1 + 4) % 5][c1];
        *y = matrix[(r2 + 4) % 5][c2];
    }
}
else {
    *x = matrix[r1][c2];
    *y = matrix[r2][c1];
}

```

```

}

void prepareText(char *input, char *output) { int k = 0;

```

```

for (int i = 0; input[i] != '\0'; i++) {
    if (isalpha(input[i])) {
        char ch = toupper(input[i]);

        if (ch == 'J')
            ch = 'I';

        output[k++] = ch;
    }
}

```

```

output[k] = '\0';

```

```

}

```

```

void encryptText(char *input, char *output) { char text[1000]; prepareText(input, text);

```

```

int len = strlen(text);
int k = 0;

```

```

for (int i = 0; i < len;) {
    char a = text[i];
    char b;

```

```

    if (i + 1 >= len) {
        b = 'X';
        i++;
    }
    else if (text[i] == text[i + 1]) {
        b = 'X';
        i++;
    }
    else {
        b = text[i + 1];
        i += 2;
    }

    processPair(a, b, &output[k], &output[k + 1], 1);
    k += 2;
}

output[k] = '\0';

}

void decryptText(char *input, char *output) { int len = strlen(input); int k = 0;

for (int i = 0; i < len; i += 2) {
    processPair(input[i], input[i + 1],
        &output[k], &output[k + 1], 0);
    k += 2;
}

output[k] = '\0';

}

void displayMatrix() { printf("\nPlayfair Matrix:\n");

```

```

for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        printf("%c ", matrix[i][j]);
    }
    printf("\n");
}

}

int main() { char plaintext[1000]; char key[100]; char ciphertext[1000]; char
decrypted[1000];

printf("Enter plaintext: ");
fgets(plaintext, sizeof(plaintext), stdin);

printf("Enter keyword: ");
fgets(key, sizeof(key), stdin);

generateMatrix(key);

displayMatrix();

encryptText(plaintext, ciphertext);
printf("\nEncrypted message: %s\n", ciphertext);

decryptText(ciphertext, decrypted);
printf("Decrypted message: %s\n", decrypted);

return 0;

}

```

Input & Output :

Enter plaintext: HELLO
Enter keyword: MONARCHY

Playfair Matrix:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Encrypted message: CFSUPM

Decrypted message: HELXLO